

Comunicação entre Componentes

📁 Arquivo: `filho.ts`

Caminho: `Projeto04_Comunicacao/src/app/components/filho/filho.ts`

```
TS filho.ts U x  </> filho.html U  TS app.ts U  </> app.html U
Projeto04_Comunicacao > src > app > components > filho > TS filho.ts > ...
1  import { Component, input, output } from '@angular/core'; // importar input e output
2
3  @Component({
4    selector: 'app-filho',
5    standalone: true,
6    templateUrl: './filho.html',
7  })
8  export class Filho {
9
10     // @Input moderno (v20+): Recebe a informação do componente pai
11     nomeRecebido = input<string>('');
12
13     // @Output moderno (v20+): Envia informação para o componente pai
14     notificarReset = output<void>();
15
16     resetar() {
17       // Emite o evento para o pai
18       this.notificarReset.emit();
19     }
20
21   }
22
```

1. Definição de Canais de Comunicação

O componente Filho estabelece dois canais fundamentais para a troca de dados:

- **input**: Este recurso é utilizado para **receber informações do componente pai**. Na prática, ele funciona como uma porta de entrada que aceita dados (neste caso, uma *string*) injetados externamente.
- **output**: Este recurso é utilizado para **enviar informações para o componente pai**. Ele não envia o dado diretamente para uma variável do pai, mas sim dispara um evento que o pai deve "ouvir".

2. Ação e Emissão de Eventos

A função `resetar()` exemplifica o controle de fluxo ascendente:

- **this.notificarReset.emit():** Ao ser executado, este comando emite um sinal para o componente superior. Sob a ótica de engenharia, isso mantém o componente Filho "burro", ou seja, ele sabe que algo deve ser resetado, mas não possui a lógica para resetar o dado; ele apenas delega essa tarefa ao Pai.

📁 Arquivo: `filho.html`

Caminho: `Projeto04_Comunicacao/src/app/components/filho/filho.html`



```
TS filho.ts U  </> filho.html U X  TS app.ts U  </> app.html U
Projeto04_Comunicacao > src > app > components > filho > </> filho.html > ...
Go to component
1 <div style="border: 2px dashed gray; padding: 10px;">
2   <h3>Componente Filho</h3>
3   <p>Dado vindo do Pai: <strong>{{ nomeRecebido() }}</strong></p>
4
5   <button (click)="resetar()">Resetar no Pai</button>
6 </div>
7 | Ctrl+L for Agent
```

O template do componente filho é responsável por exibir os dados injetados pelo pai e capturar as interações do usuário.

1. Exibição via Interpolação: `{{ nomeRecebido() }}`

Este recurso é utilizado para renderizar o dado dinâmico na tela.

- **Mecânica:** O HTML "consome" o valor atual do signal `nomeRecebido`. Por ser um signal, os parênteses `()` são obrigatórios para extrair o valor.
- **Reatividade:** Sempre que o componente Pai atualizar o nome no "back-end", a interpolação garantirá que o novo texto apareça instantaneamente para o usuário no Filho.

2. Captura de Evento: `(click)="resetar()"`

O evento de clique no botão serve como o gatilho para a comunicação ascendente.

- **Mecânica:** Através do **Event Binding**, o HTML escuta o clique do mouse e dispara a execução da função `resetar()` presente no arquivo `filho.ts`.

- **Fluxo de Trabalho:** Esta ação inicia a cadeia de eventos: o clique ativa a função no TS do Filho, que por sua vez emite o `output` para o componente Pai realizar a limpeza do dado.

📁 Arquivo: `pai.html`

Caminho: `Projeto04_Comunicacao/src/app/app.component.html`

O componente pai funciona como o "Orquestrador" do sistema. É nele que as informações residem originalmente e onde se define como elas serão distribuídas para os componentes menores.

1. O Pai como Fonte da Verdade

Diferente do componente filho, que apenas recebe ordens, o arquivo Pai detém o dado original (o Signal `usuarioNome`). No HTML, o valor desse sinal é exibido diretamente através da **interpolação** `{{ usuarioNome() }}`, garantindo que o topo da hierarquia sempre mostre o estado real do sistema.

2. Passagem de Informação: `[nomeRecebido]`

A comunicação inicia-se na tag customizada. Ao escrever `[nomeRecebido]="usuarioNome()",` o desenvolvedor está explicitamente enviando o dado do Pai para o Filho.

- **Mecânica:** O "back-end" do componente pai (TS) fornece o valor, e o HTML atua como a tubulação que empurra esse dado para dentro do componente dependente.

3. Escuta e Reação: `(notificarReset)`

O Pai não apenas envia dados, mas também monitora o comportamento do Filho.

- **Mecânica:** A sintaxe entre parênteses indica que o Pai está em "modo de escuta". Quando o Filho dispara o sinal de reset, o Pai intercepta essa notificação e executa a função `limparNome()`, alterando o seu próprio Signal e reiniciando o ciclo de dados.

```
ts filho.ts U <> filho.html U ts app.ts U <> app.html U x
Projeto04_Comunicacao > src > app > <> app.html > ...
Go to component
1 <main style="padding: 20px;">
2 <h1>Projeto 04: Comunicação entre Componentes</h1>
3 <p>Nome no Pai: {{ usuarioNome() }}</p>
4
5 <hr>
6 <!--Essa tag demonstra a comunicação entre o componente pai e filho-->
7 <app-filho
8   [nomeRecebido]="usuarioNome()"
9   (notificarReset)="limparNome()"
10 </app-filho>
11 </main>
12
13 <!--
14 nomeRecebido e nomeRecebido são atributos ou variáveis vindo graças a importação vindo de app.ts (componente pai)
```

📁 Arquivo: `pai.html`

Caminho: `Projeto04_Comunicacao/src/app/app.component.html`

O código neste arquivo representa o ponto de encontro entre a lógica global da aplicação e os módulos especializados (componentes filhos).

1. A Tag `<app-filho>` e o Isolamento de Escopo

A presença da tag `<app-filho>` `</app-filho>` indica que está ocorrendo uma **instanciação** do componente.

- **Escopo Fechado:** Esta tag garante que a aplicação dentro dela esteja isolada. Isso significa que as variáveis e o CSS do componente filho não interferem no restante do código do pai, mantendo a integridade e a organização do software.
- **Sinal de Comunicação:** Sempre que esta tag é utilizada com propriedades entre colchetes ou parênteses, confirma-se que há uma troca de informações entre o componente pai e o filho.

2. Atributos e Variáveis de Importação

Os nomes como `nomeRecebido` são atributos ou variáveis que existem no componente filho graças à estrutura definida no `app.ts` (ou `filho.ts`).

- **Conexão:** O HTML do pai utiliza esses nomes para saber exatamente onde "entregar" os dados ou onde "ouvir" os eventos.

3. Protocolos de Binding

A comunicação é regida por duas sintaxes fundamentais:

- **[propriedadeBinding]="valor"**: Utilizado para enviar informações do pai para o filho. O "valor" (geralmente um Signal do pai) é injetado na propriedade do filho.
- **(eventoBinding)="funcao()"**: Utilizado para receber notificações do filho. Quando o filho dispara um evento, o pai executa a função correspondente.



Resumo Geral: Fluxo de Comunicação Inter-Componentes

A comunicação no Angular fundamenta-se no princípio de **Responsabilidade Única** e **Desacoplamento**. O sistema é dividido em dois papéis principais que trocam informações através de protocolos específicos.

1. O Papel do Componente Pai (Gestor)

O componente pai atua como a "Fonte da Verdade" e o orquestrador dos dados.

- **Envio de Dados (Property Binding)**: Utiliza a sintaxe de colchetes [] para injetar valores (Signals) no componente filho.
- **Escuta de Ações (Event Binding)**: Utiliza a sintaxe de parênteses () para monitorar eventos emitidos pelo filho e executar funções de resposta (como resetar ou atualizar dados).

2. O Papel do Componente Filho (Módulo)

O componente filho funciona como uma unidade isolada e reutilizável que reage a entradas e reporta atividades.

- **Recepção (input)**: Declara atributos que servem como portas de entrada para informações vindas do pai.
- **Notificação (output)**: Utiliza um emissor de eventos para "avisar" o pai que uma interação (como um clique) ocorreu, sem precisar conhecer a lógica interna do pai.

 Tabela de Referência Rápida

Direção	Sintaxe	Recurso Angular	Analogia de Engenharia
Descendente (<i>Pai</i> → <i>Filho</i>)	[prop]	@Input() / input()	Parâmetro de entrada de um método. 
Ascendente (<i>Filho</i> → <i>Pai</i>)	(event)	@Output() / output()	Disparo de Evento (Trigger) / Callback  +1